

няет другие финализирующие задачи. В нашем примере приложения для чата это может включать обновление ввода сообщения, отображение или скрытие индикатора ввода и добавление новых сообщений в список сообщений. Затем React переходит к следующему циклу рендеринга, повторяя процесс сбора обновлений, обработки полос и фиксации изменений.

Однако этот процесс значительно сложнее, чем мы можем по-настоящему оценить в этой книге: существуют такие концепции, как *"запутанность"* (*entanglement*), которая определяет, когда две полосы должны обрабатываться совместно. Другая концепция, например *"перебазирование"* (*rebasing*), определяет, когда обновление должно быть перебазировано поверх обновлений, которые уже были обработаны. Перебазирование полезно, например, в тех случаях, когда переход (*transition*) прерывается до его завершения из-за обновления синхронизации, и вам нужно запустить вместе оба обновления.

Здесь также можно многое сказать об эффектах сброса (*flushing effects*). Например, при синхронном обновлении React может сбрасывать эффекты до или после обновления, чтобы обеспечить согласованное состояние синхронизированных обновлений.

В конечном счете именно для этого существует React, и истинная ценность React заключается в том, что он работает за кулисами как уровень абстракции: он в основном решает проблемы с обновлениями, их приоритезацией и упорядочиванием за нас, в то время как мы продолжаем фокусироваться на наших приложениях.

Важно отметить, что, хотя React хорошо оценивает приоритеты, он не всегда идеален. Как разработчику, вам иногда может потребоваться изменить приоритет задания по умолчанию, используя некоторые API, о которых мы уже упоминали — `useTransition` и `useDeferredValue` — для точной настройки производительности и отзывчивости вашего приложения. Давайте рассмотрим эти API более подробно.

useTransition

`useTransition` — это мощный хук React, который позволяет вам управлять приоритетом обновлений состояния ваших компонентов и гарантировать, что пользовательский интерфейс останется отзывчивым в случае высокоприоритетных обновлений. Это особенно полезно, когда приходится иметь дело с обновлениями, которые могут нарушить отрисовку, например, при загрузке новых данных или навигации между страницами.

По сути, любое обновление, которое вы оборачиваете в возвращаемую функцию `startTransition`, помещается в полосу перехода (*transition lane*), которая, как мы видели, имеет более низкий приоритет, чем полоса синхронизации (*Sync lane*). Это позволяет контролировать время обновления и поддерживать бесперебойную рабо-

ту пользователя, даже когда другие обновления с более высоким приоритетом конкурируют за доступ к основному потоку.

`useTransition` — это хук, который можно использовать только внутри функциональных компонентов. Он возвращает массив, содержащий два элемента.

`isPending`

Логическое значение, указывающее, выполняется ли переход. Одна интересная деталь работы `useTransition` заключается в том, что первое, что он делает, когда вы вызываете `startTransition`, — это установка `setState({ isPending: false })`, означающая, что обновления, зависящие от `isPending`, должны быть быстрыми, иначе это противоречит цели `useTransition`.

`startTransition`

Функция, которую можно использовать для оборачивания обновлений, которые следует отложить или которым следует присвоить более низкий приоритет.

Вероятно, здесь стоит упомянуть, что существует также API `startTransition`, который доступен не как хук, а как *обычная функция*. Второй способ запустить несрочный переход — использовать функцию `startTransition`, импортированную непосредственно из `React`. Этот подход не дает нам доступа к флагу `isPending`, но он доступен для тех мест в вашем коде, где вы не можете использовать хуки, например `useTransition`, но все равно хотите сигнализировать `React` об обновлении с низким приоритетом.

Простой пример

Вот простой пример, демонстрирующий основные принципы использования `useTransition`:

```
import React, { useState, useTransition } from "react";
```

```
function App() {
  const [count, setCount] = useState(0);
  const [isPending, startTransition] = useTransition();

  const handleClick = () => {
    doSomethingImportant();
    startTransition(() => {
      setCount(count + 1);
    });
  };

  return (
    <div>
      <p>Count: {count}</p>
```

```

    <button onClick={handleClick}>Increment</button>
    {isPending && <p>Loading...</p>}
  </div>
);
}

```

```
export default App;
```

В этом примере мы используем `useTransition` для управления приоритетом обновления состояния, которое увеличивает счетчик. Помещая обновление `setCount` в функцию `startTransition`, мы сообщаем React, что это обновление может быть отложено, если одновременно происходят другие высокоприоритетные обновления. Цель в том, чтобы пользовательский интерфейс не перестал отвечать на запросы.

Более сложный пример: навигация

`useTransition` также полезен при организации переходов между страницами. Управляя приоритетом обновлений, связанных с навигацией, вы можете обеспечить плавность и отзывчивость при взаимодействии с пользователем даже в случае сложных переходов по страницам.

Рассмотрим пример, в котором мы продемонстрируем, как использовать `useTransition` для управления переходами в одностраничном приложении (single-page application, SPA):

```
import React, { useState, useTransition } from "react";
```

```
const PageOne = () => <div>Page One</div>;
```

```
const PageTwo = () => <div>Page Two</div>;
```

```
function App() {
  const [currentPage, setCurrentPage] = useState("pageOne");
  const [isPending, startTransition] = useTransition();

```

```

  const handleNavigation = (page) => {
    startTransition(() => {
      setCurrentPage(page);
    });
  };
}

```

```

const renderPage = () => {
  switch (currentPage) {
    case "pageOne":
      return <PageOne />;

```

```

    case "pageTwo":
      return <PageTwo />;
    default:
      return <div>Unknown page</div>;
  }
};

return (
  <div>
    <nav>
      <button onClick={() => handleNavigation("pageOne")}>Page One</button>
      <button onClick={() => handleNavigation("pageTwo")}>Page Two</button>
    </nav>
    {isPending && <p>Loading...</p>}
    {renderPage()}
  </div>
);
}

export default App;

```

В этом примере у нас есть два простых компонента, представляющих разные страницы в нашем SPA. Мы используем `useTransition` для оборачивания обновления состояния, которое изменяет текущую страницу, гарантируя, что переход страницы будет отложен при наличии других высокоприоритетных обновлений (например, ввод данных пользователем).

Вы можете подумать: "Подождите, разве переход на страницу не должен быть мгновенным, поскольку он происходит в ответ на клик пользователя?" Да, вы были бы правы; однако, если для перехода на следующую страницу требуется подгрузить некоторые данные с помощью `Suspense`, тогда переход на другую страницу может быть отложен. Именно здесь пригодится `useTransition`, поскольку он позволяет вам управлять приоритетом обновлений, связанных с навигацией, гарантируя, что взаимодействие с пользователем остается плавным и отзывчивым. Стоит отметить, что если выборка данных на следующей странице происходит в эффekte, то `startTransition` не будет ждать, пока будут выбраны данные. Однако, когда вы остановитесь внутри перехода, React свяжет состояние `isPending` с выборкой данных и отображением этих данных, когда вы вернетесь обратно.

В этом случае во время перехода на другую страницу состояние `isPending` будет равно `true`, что позволяет нам немедленно показать индикатор загрузки пользователю в ответ на его нажатие кнопки. Как только переход будет завершен, состояние `isPending` станет равно `false`, и будет отображена новая страница.

Погружаемся глубже

Благодаря базовым знаниям о Fiber-архитектуре React, планировщике React, уровнях приоритета и механизме полос рендеринга теперь мы можем разобраться во внутренней работе хука `useTransition`.

Механизм `useTransition` работает путем создания перехода и присвоения определенного уровня приоритета обновлениям, выполняемым в рамках этого перехода. Когда обновление обернуто в переход, React гарантирует, что обновление будет запланировано и отрисовано в соответствии с назначенным уровнем приоритета.

Вот краткий обзор шагов, связанных с использованием хука `useTransition`:

1. Импорт и вызов хука `useTransition` внутри функционального компонента.
2. Хук возвращает массив с двумя элементами: первый — это состояние `isPending`, а второй — функция `startTransition`.
3. Функция `startTransition` используется для оборачивания любого обновления состояния или рендеринга компонента, тайминг которого вы хотите контролировать.
4. Состояние `isPending` показывает, продолжается ли переход или он завершен.
5. React гарантирует, что обновления, обернутые в переход, обрабатываются с соответствующим уровнем приоритета. Это достигается за счет использования планировщика и механизма полос рендеринга для назначения обновлений и управления ими.

Используя `useTransition`, мы можем эффективно контролировать тайминг обновления и обеспечивать бесперебойную работу пользователей, даже когда другие обновления с более высоким приоритетом конкурируют за доступ к основному потоку.

useDeferredValue

`useDeferredValue` — это хук React, который позволяет отложить определенные обновления пользовательского интерфейса на более позднее время, что особенно полезно в сценариях, когда приложение сталкивается с большой нагрузкой или задачами, требующими больших вычислительных затрат. Таким образом, этот хук помогает управлять приоритезацией обновлений и обеспечивает более плавные переходы и улучшенный пользовательский опыт.

Во время первоначального рендеринга возвращаемое отложенное значение совпадает с заданным значением. В последующих обновлениях `useDeferredValue` помогает поддерживать беспрепятственную работу с пользователем, сохраняя старое значение в течение более длительного времени перед обновлением до нового значения, особенно в сценариях с интенсивными вычислительными операциями. Это не приводит к многократному повторному рендерингу со старыми и новыми значениями, а влечет за собой контролируемое обновление до нового значения. Этот механизм

сродни стратегии `stale-while-revalidate`, при которой сохраняются устаревшие значения, чтобы пользовательский интерфейс оставался отзывчивым в ожидании новых значений.

Просматривая историю версий React, мы видим, что первая реализация `useDeferredValue` выглядела примерно так:

```
function useDeferredValue(value) {  
  const [newValue, setNewValue] = useState(value); // сохраняет только  
                                                    // начальное значение  
  
  useEffect(() {  
    // обновляет возвращаемое значение при переходе всякий раз,  
    // когда оно изменяется, "откладывая" его  
    startTransition(() => {  
      setNewValue(value);  
    });  
  }, [value]);  
  
  return newValue;  
}
```

Давайте немного поговорим о том, что делает этот код. Сначала он устанавливает состояние (`newValue`) с переданным ему начальным значением. Затем функция использует хук `useEffect` для наблюдения за изменениями этого значения. При обнаружении изменения вызывается функция `startTransition`, которая имеет решающее значение для отсрочки обновления.

В `startTransition` состояние обновляется до нового значения с помощью `setNewValue`. Использование `startTransition` сигнализирует React о том, что это обновление не является срочным, что позволяет React сначала определить приоритетность других, более важных обновлений. Это более или менее точная модель работы `useDeferredValue` сегодня, которая должна быть полезна для нашего понимания функционирования этого хука.

`useDeferredValue` — это часть конкурентных особенностей React, которая обеспечивает возможность прерывания, позволяя откладывать некоторые обновления состояния.

Когда компонент повторно отображается с отложенным значением, React продолжает показывать старое значение в течение некоторого периода времени, позволяя обновлениям с высоким приоритетом обрабатываться раньше, чем обновлениям с низким приоритетом. Это разбивает работу по рендерингу на более мелкие фрагменты, которые можно распределять во времени, повышая скорость отклика и обеспечивая высокий приоритет обновления (например, взаимодействие с пользователем). Высокоприоритетные обновления не задерживаются из-за обновлений с более низким приоритетом, что повышает позитивный пользовательский опыт.

Цель использования *useDeferredValue*

Основная цель *useDeferredValue* — позволить вам отложить вывод менее важных обновлений. Это особенно полезно, когда вы хотите расставить приоритеты между более важными обновлениями, такими как взаимодействие с пользователем, и менее важными, такими как отображение обновленных данных с сервера.

Применяя *useDeferredValue*, вы можете обеспечить более плавный пользовательский интерфейс и гарантировать, что ваше приложение останется отзывчивым даже при большой нагрузке или при выполнении сложных операций.

Для того чтобы использовать *useDeferredValue*, вам нужно импортировать его из пакета *React* и передать значение, которое будет отложено, в качестве аргумента. Затем хук вернет отложенную версию значения, которую можно использовать в вашем компоненте.

Вот пример того, как использовать *useDeferredValue* в простом приложении:

```
import React, { memo, useState, useDeferredValue } from "react";

function App() {
  const [searchValue, setSearchValue] = useState("");
  const deferredSearchValue = useDeferredValue(searchValue);

  return (
    <div>
      <input
        type="text"
        value={searchValue}
        onChange={(event) => setSearchValue(event.target.value)}
      />
      <SearchResults searchValue={deferredSearchValue} />
    </div>
  );
}

const SearchResults = memo(({ searchValue }) => {
  // Производим поиск и отображаем результаты
})
```

В этом примере у нас есть ввод для поиска и компонент *SearchResults*, который отображает результаты. Мы используем *useDeferredValue* для отсрочки вывода результатов поиска, что позволяет приложению определять приоритеты ввода пользователем и оставаться отзывчивым, даже если вывод списка результатов является дорогостоящим процессом.

Давайте разберемся в этом чуть более подробно:

1. Мы используем `memo`, чтобы убедиться, что компонент не обновляется без необходимости, как мы обсуждали в предыдущих главах.
2. Когда компонент обновляется, это вызывает проблемы с производительностью, поскольку его рендеринг обходится дорого.
3. Когда мы присваиваем компоненту отложенный пропс, `deferredSearchValue`, поскольку сам пропс обновляется после более срочной работы по рендерингу, обновление происходит и с компонентом. Таким образом, компонент повторно отображается только тогда, когда нет более срочной работы, например обновления поля ввода текста.

Кто-то может спросить: "Почему бы просто не отменить (`debounce`) или не ограничить (`throttle`) значение `searchValue`?"

Отличный вопрос. Давайте сравним эти действия.

Отмена.

Предполагает паузу перед обновлением списка, для ожидания, пока пользователь закончит ввод текста, например, задержку в одну секунду

Ограничение.

Обновляет список через регулярные промежутки времени, скажем, не чаще одного раза в секунду

Хотя эти методы могут быть эффективны в определенных ситуациях, `useDeferredValue` представляется более подходящим решением для оптимизации рендеринга, поскольку оно легко адаптируется к возможностям производительности устройства пользователя и не вызывает случайные задержки.

Ключевое отличие в случае `useDeferredValue` заключается в динамическом подходе к задержкам. Это устраняет необходимость в установке фиксированного времени задержки. На высокопроизводительных устройствах, таких как мощный ноутбук, задержка при повторной визуализации незаметна и происходит практически мгновенно. И наоборот, на более медленных устройствах задержка рендеринга регулируется в соответствии с производительностью, и список обновляется в ответ на ввод с небольшой задержкой, пропорциональной скорости устройства.

Кроме того, `useDeferredValue` обладает значительным преимуществом в возможности прерывать отложенные повторные рендеринги. В сценариях, когда React обрабатывает большой список, а пользователь совершает новое нажатие клавиши, React может приостановить повторную визуализацию, отреагировать на новый ввод данных, а затем возобновить процесс рендеринга в фоновом режиме. Это отличается от *отмены* и *ограничений*, которые, несмотря на задержку обновления, все равно могут привести к сбоям в работе, поскольку блокируют интерактивность во время рендеринга.

Тем не менее отмена и ограничение по-прежнему полезны в сценариях, не связанных напрямую с рендерингом. Например, они могут быть эффективны для снижения частоты сетевых запросов. Эти методы также могут быть использованы в сочетании с `useDeferredValue` для комплексной стратегии оптимизации.

Исходя из всего этого, мы видим несколько преимуществ использования `useDeferredValue` в приложениях React.

Улучшена скорость отклика.

В примере, когда пользователь вводит текст в поле поиска, поле ввода немедленно обновляется, а результаты отображаются с задержкой. Если пользователь быстро вводит пять символов подряд, поле ввода обновляется после ввода каждого символа, а `searchResults` отображается только один раз, после того как пользователь прекратит вводить текст. Для символов 1–4 отображение `searchResults` прерывается вводом новых значений.

Декларативная расстановка приоритетов.

`useDeferredValue` обеспечивает простой и декларативный способ управления приоритетизацией обновлений в вашем приложении. Инкапсулируя логику отсрочки обновлений внутри хука, вы можете сохранить чистоту кода компонента и сфокусироваться на важных аспектах вашего приложения.

Более эффективное использование ресурсов.

Благодаря отсрочке менее важных обновлений хук `useDeferredValue` позволяет вашему приложению более эффективно использовать доступные ресурсы. Это может помочь снизить вероятность возникновения "узких мест" и повысить общую производительность вашего приложения.

Когда применять `useDeferredValue`

`useDeferredValue` наиболее полезен в ситуациях, когда вашему приложению необходимо отдавать предпочтение определенным обновлениям перед другими. Вот несколько распространенных сценариев, в которых можно использовать `useDeferredValue`:

- ◆ поиск или фильтрация больших наборов данных;
- ◆ визуализация сложных компонентов или анимаций;
- ◆ обновление данных с сервера в фоновом режиме;
- ◆ выполнение дорогостоящих вычислительных операций, которые могут повлиять на взаимодействие с пользователем.

Давайте рассмотрим пример, в котором `useDeferredValue` может быть особенно полезным. Представьте, что у нас есть большой список элементов, которые мы хотим отфильтровать на основе пользовательского ввода. Фильтрация длинного списка может потребовать больших вычислительных затрат, поэтому использование `useDeferredValue` может помочь сохранить отзывчивость приложения:

```
import React, { memo, useState, useMemo, useDeferredValue } from "react";
```

```
function App() {
  const [filter, setFilter] = useState("");
  const deferredFilter = useDeferredValue(filter);

  const items = useMemo(() => generateLargeListOfItems(), []);
```

```

const filteredItems = useMemo(() => {
  return items.filter((item) => item.includes(deferredFilter));
}, [items, deferredFilter]);

return (
  <div>
    <input
      type="text"
      value={filter}
      onChange={(event) => setFilter(event.target.value)}
    />
    <ItemList items={filteredItems} />
  </div>
);
}

const ItemList = мемо(({ items }) => {
  // Рендеринг списка
});

function generateLargelistOfItems() {
  // Генерация большого списка для примера
}

```

В этом примере мы используем `useDeferredValue` для отсрочки вывода отфильтрованного списка. По мере того, как пользователь вводит данные для фильтра, отложенное значение обновляется реже, что позволяет приложению отдавать приоритет пользовательскому вводу и оставаться отзывчивым.

Хуки `useMemo` используются для запоминания элементов и массивов `filteredItems`, предотвращая ненужные повторные визуализации и вычисления. Это еще больше повышает производительность приложения.

Когда не следует использовать `useDeferredValue`

Хотя в определенных сценариях `useDeferredValue` может быть полезным, важно учитывать компромиссы. А именно, при отсрочке обновления существует вероятность того, что данные, показываемые пользователю, могут немного устареть. Хотя это обычно приемлемо для менее важных обновлений, необходимо учитывать последствия отображения устаревших данных для пользователей.

При принятии решения о том, следует ли использовать `useDeferredValue` или нет, нужно задать себе вопрос: "Является ли это обновление результатом введения данных пользователем?"

React называется React не просто так: он позволяет нашим веб-приложениям реагировать на различные события. Все, что побуждает пользователя ожидать реакции, не должно откладываться. При этом все остальное должно быть отложено.

Хотя использование `useDeferredValue` может значительно повысить скорость реагирования вашего приложения при увеличенной нагрузке, его не следует рассматривать как волшебную палочку. Всегда помните, что лучший способ повысить производительность — это написать эффективный код и избежать ненужных вычислений.

Проблемы с конкурентным рендерингом

Конкурентный рендеринг, хотя и обеспечивает эффективное и отзывчивое взаимодействие с пользователем, создает новые проблемы для разработчиков. Основная проблема заключается в том, что трудно определить правильный порядок обработки обновлений, что может привести к неожиданному поведению и ошибкам.

Одна из таких ошибок называется *разрывом* (tearing), когда пользовательский интерфейс становится несогласованным из-за того, что обновления обрабатываются не по порядку. Это может произойти, когда компонент зависит от некоторого значения, обновляемого во время его отрисовки, что приводит к отрисовке приложений с противоречивыми данными. Давайте немного углубимся в это.

Разрыв

Разрыв — это ошибка, возникающая, когда компонент зависит от некоторого состояния, которое обновляется, пока приложение все еще выполняет рендеринг. Для того чтобы понять это, давайте сравним синхронный рендеринг с конкурентным рендерингом.

В синхронном мире React просматривал бы дерево компонентов и отображал бы их один за другим, сверху вниз. Это гарантирует, что состояние приложения остается неизменным на протяжении всего процесса рендеринга, поскольку каждый компонент отображается в последнем состоянии.

Рассмотрим такой пример:

```
import { useState, useSyncExternalStore, useTransition } from "react";
```

```
// Внешнее состояние
```

```
let count = 0;
```

```
setInterval(() => count++, 1);
```

```
export default function App() {
```

```
  const [name, setName] = useState("");
```

```
  const [isPending, startTransition] = useTransition();
```

```
  const updateName = (newVal) => {
```

```
    startTransition(() => {
```

```

    setName(newVal);
  });
};

return (
  <div>
    <input value={name} onChange={(e) => updateName(e.target.value)} />
    {isPending && <div>Loading...</div>}
    <ul>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
    </ul>
  </div>
);
}

const ExpensiveComponent = () => {
  const now = performance.now();

  while (performance.now() - now < 100) {
    // Ничего не делаем, просто ждем.
  }

  return <>Expensive count is {count}</>;
};

```

В самом верху нашего приложения у нас есть `count` — переменная, которую мы устанавливаем глобально и постоянно обновляем с помощью `setInterval` вне цикла

рендеринга React, чтобы мы могли имитировать ошибку разрыва, обновляя ее во время рендеринга приложения. Поскольку рендеринг конкурентный и может прерываться, возможно, что `ExpensiveComponent` будет отображаться с разными значениями `count`, что приведет к отображению несогласованных данных для пользователя или их разрыву.

Мы ожидаем увидеть несогласованные значения `count`, отображаемые внутри `ExpensiveComponent`, поскольку React "останавливает" рендеринг при вводе пользователем данных, чтобы определить приоритет более срочного обновления, такого как обновление поля ввода текста, тем самым оставляя устаревшее значение `count` в `ExpensiveComponent`, но не каждый раз, только иногда.

В нашем примере отображается поле ввода текста и список из пяти экземпляров `ExpensiveComponent`. Эти компоненты специально не запоминаются, чтобы проиллюстрировать суть, поскольку они вызывают проблемы с производительностью, а нам нужны эти проблемы, чтобы идентифицировать разрывы в целях их понимания. В реальном мире вам захочется обернуть `ExpensiveComponent` в `React.memo`. Здесь мы намеренно избегаем этого, чтобы продемонстрировать разрывы, которых вы захотите избежать в своем приложении.

Рендеринг `ExpensiveComponent` занимает много времени, имитируя вычислительный процесс дорогостоящей операции. `ExpensiveComponent` также отображает текущее значение переменной `count`, которое увеличивается каждую миллисекунду и считывается из внешнего хранилища, в данном случае из глобального пространства имен.

Если мы запустим этот пример, то увидим, что для пяти экземпляров `ExpensiveComponent`, которые мы визуализируем, после ввода нескольких нажатий клавиш компоненты `ExpensiveComponent` будут отображаться с разными значениями `count`.

Это происходит потому, что `ExpensiveComponent` визуализируется пять раз, и каждый раз, когда он визуализируется, значения `count` отличаются. Поскольку React отображает компоненты конкурентно, возможно, что `ExpensiveComponent` будет отображаться с разными значениями `count`, что приведет к несогласованным данным, отображаемым пользователю.

Это называется разрывом, и это ошибка, которая может возникнуть, когда компонент зависит от некоторого состояния, обновляемого во время процесса рендеринга приложения. В этом случае `ExpensiveComponent` зависит от переменной `count`, которая обновляется во время выполнения рендеринга компонента, что приводит к рендерингу приложения с несогласованными данными. При разрыве для пяти экземпляров `ExpensiveComponent` мы видим следующий результат:

- ◆ `Expensive count is 568;`
- ◆ `Expensive count is 568;`
- ◆ `Expensive count is 569;`
- ◆ `Expensive count is 569;`
- ◆ `Expensive count is 570.`

Все верно, потому что отображаются более ранние экземпляры компонента, обновленное значение `count` сбрасывается/фиксируется в DOM, а более ранние экземпляры продолжают отображаться и выдаются (сбрасываются, обновляются) с более новыми значениями `count`.

Ситуация не критична, потому что React в конечном итоге отображает согласованное состояние. Более серьезная проблема возникает, когда у вас есть что-то вроде:

```
<UserDetails id={user.id} />
```

Этот код, если пользователь будет удален из глобального хранилища между отрисовками страницы, приведет к внезапной ошибке, которая может удивить пользователя. Вот почему разрыв является проблемой.

Для того чтобы решить эту проблему, React предоставляет хук под названием `useSyncExternalStore`. Давайте разберемся с этим хуком.

useSyncExternalStore

`useSyncExternalStore` — это хук React, который позволяет синхронизировать внешнее состояние с внутренним состоянием вашего приложения. Это особенно полезно при выполнении дорогостоящих вычислительных операций, которые могут привести к разрыву при неправильной обработке. Термин "синхронизация" в `useSyncExternalStore` имеет двойное значение. Он означает и "синхронизировать", и "синхронный": он запускает синхронное обновление при изменении хранилища.

Механизм использования внешнего хранилища для синхронизации имеет следующую сигнатуру:

```
const value = useSyncExternalStore(store.subscribe, store.getSnapshot);
```

`store.subscribe`

Это функция, которая получает функцию обратного вызова в качестве своего первого и единственного аргумента. Внутри этой функции вы можете подписаться на изменения во внешнем хранилище и вызывать функцию обратного вызова при каждом изменении в хранилище. Обратный вызов можно рассматривать как вызов, позволяющий оперативно отреагировать на повторную отрисовку компонента с новым значением. Ожидаемый результат этой функции — функция очистки, которая отменяет подписку на хранилище.

Типичная функция подписки `subscribe` выглядит следующим образом:

```
const store = {
  subscribe(render) {
    const newData = getNewData().then(render);
    return () => {
      // отписаться как-нибудь
    };
  },
};
```

Простым вариантом использования для этого была бы подписка на события браузера, такие как изменение размера `resize` или прокрутка `scroll`. Обновление компонента при возникновении этих событий можно отразить следующим образом:

```
const store = {
  subscribe(rerenderImmediately) {
    window.addEventListener("resize", rerenderImmediately);
    return () => {
      window.removeEventListener("resize", rerenderImmediately);
    };
  },
};
```

Теперь наши компоненты React будут повторно отрисовываться при изменении размера окна браузера. Однако как они получают новое значение? Вот здесь появляется второй аргумент для `useSyncExternalStore`.

`store.getSnapshot`

Функция, которая возвращает текущее значение внешнего хранилища. Она вызывается при каждом отображении компонента, а возвращаемое значение используется для обновления внутреннего состояния компонента. Эта функция вызывается синхронно, поэтому она не должна выполнять никаких асинхронных операций или иметь каких-либо побочных эффектов. Более того, эта функция гарантирует, что состояние во время рендеринга будет одинаковым для нескольких экземпляров компонента.

Следуя нашему примеру с изменением размера окна, мы получим текущий размер окна следующим образом:

```
const store = {
  subscribe(immediatelyRerenderSynchronously) {
    window.addEventListener("resize",
      immediatelyRerenderSynchronously);
    return () => {
      window.removeEventListener("resize",
        immediatelyRerenderSynchronously);
    };
  },
  getSnapshot() {
    return {
      width: window.innerWidth,
      height: window.innerHeight,
    };
  },
};
```

Объект с параметрами { width, height } — это моментальный снимок текущего состояния окна, и именно его вернет useSyncExternalStore. Затем мы можем использовать этот объект в нашем компоненте с уверенностью, что при конкурентном рендеринге его состояние всегда будет одинаковым.

Откуда у нас такая уверенность? Это потому, что функция immediatelyRenderSynchronously запускает синхронную повторную отрисовку и не позволяет React отложить ее. Это ключ к решению проблемы разрыва.

Теперь давайте посмотрим, как мы можем использовать useSyncExternalStore для решения проблемы разрыва в нашем предыдущем примере. Если мы помним, мы видели список экземпляров ExpensiveComponent, которые отображались с разными значениями count из-за разрыва. Сейчас мы посмотрим, как можно это исправить, используя синхронизацию с внешним хранилищем useSyncExternalStore.

Мы не хотим подписываться на хранилище и запускать повторную отрисовку при появлении обновлений. Вместо этого нам нужно согласованное состояние повторной отрисовки из-за пользовательского ввода. Таким образом, наша функция subscribe будет пустой, но для получения согласованного состояния мы будем использовать функцию getSnapshot, чтобы получить текущее значение параметра count и вернуть его:

```
const store = {
  subscribe() {},
  getSnapshot() {
    return count;
  },
};
```

Вот как будет выглядеть наш предыдущий пример с useSyncExternalStore:

```
import { useState, useSyncExternalStore, useTransition } from "react";
```

```
let count = 0;
setInterval(() => count++, 1);
```

```
export default function App() {
  const [name, setName] = useState("");
  const [, startTransition] = useTransition();

  const updateName = (newVal) => {
    startTransition(() => {
      setName(newVal);
    });
  };

  return (
    <div>
```



```



```

```

const ExpensiveComponent = () => {
  // Вместо глобального чтения count
  // мы применим хук, чтобы обеспечить согласованное состояние
  const consistentCount = useSyncExternalStore(
    () => {},
    () => count
  );

  const now = performance.now();
  while (performance.now() - now < 100) {
    // Ничего не делаем
  }

  return <>Expensive count is {consistentCount}</>;
};

```

Теперь, если мы запустим этот пример, мы увидим, что компоненты `ExpensiveComponent` отображаются с одинаковым значением параметра `count`, что предотвращает возникновение разрыва. Это связано с тем, что функция `useSyncExternalStorage` обеспе-

чивает согласованность состояния во время рендеринга во всех экземплярах компонента.

Мы не используем функцию `subscribe`, потому что ее цель — сообщить React, когда следует выполнить повторную отрисовку с последним состоянием, но в нашем случае мы просто хотим, чтобы состояние было согласованным во всех отрисовках. Мы используем функцию `getSnapshot`, чтобы получить текущее значение параметра `count` и вернуть его, обеспечивая согласованность состояния во время рендеринга нескольких экземпляров компонента.

Таким образом, мы можем использовать `useSyncExternalStore` для решения проблемы разрыва в нашем предыдущем примере, гарантируя, что состояние во время рендеринга будет согласовано для нескольких экземпляров компонента.

Хорошо. Это гарантирует, что при вводе текста и повторной отрисовке компонент `ExpensiveComponent` будет иметь то же значение `count`, что и другие экземпляры `ExpensiveComponent`. Это предотвращает разрыв, но что, если мы захотим обновить `count` внутри `ExpensiveComponent` с тем же интервалом, с которым мы обновляем `count` за пределами `ExpensiveComponent`?

Мы просто создаем для него хранилище, которое следует тем же правилам обновления:

```
import { useState, useSyncExternalStore, useTransition } from "react";
```

```
let count = 0;
```

```
setInterval(() => count++, 1);
```

```
const store = {
  subscribe(forceSyncRerender) {
    // Если count изменился...
    forceSyncRerender();
  },
  getSnapshot() {
    return count;
  },
};
```

```
export default function App() {
  const [name, setName] = useState("");
  const [, startTransition] = useTransition();
```

```
  const updateName = (newVal) => {
    startTransition(() => {
      setName(newVal);
    });
  };
};
```

```

return (
  <div>
    <input value={name} onChange={(e) => updateName(e.target.value)} />
    <ul>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
      <li>
        <ExpensiveComponent />
      </li>
    </ul>
  </div>
);
}

const ExpensiveComponent = () => {
  // Вместо глобального считывания count мы будем использовать хук
  // для обеспечения согласованного состояния
  const consistentCount = useSyncExternalStore(
    store.subscribe,
    store.getSnapshot
  );

  const now = performance.now();
  while (performance.now() - now < 100) {
    // Ничего не делаем
  }

  return <>Expensive count is {consistentCount}</>;
};

```

Теперь, всякий раз, когда count изменяется, ExpensiveComponent будет отображаться с новым значением count, и мы увидим одинаковое значение count во всех экземпля-

рах `ExpensiveComponent`. Сама логика обнаружения изменений может быть как простой, так и сложной, какой вы хотите ее видеть, но главное в том, что мы понимаем механизмы, с помощью которых `useSyncExternalStore` выполняет свои основные функции, а именно:

- ◆ обеспечивает согласованное состояние при конкурентном рендеринге;
- ◆ принудительно выполняет синхронный повторный рендеринг при изменении хранилища.

Теперь, когда мы поняли, как `useSyncExternalStore` работает и решает проблему разрыва, мы научились разбираться не только в конкурентном рендеринге в React, но и в том, как решать некоторые возникающие при этом проблемы. Это важная область для приложения ваших усилий в качестве разработчика React.

Это было довольно глубокое погружение, но мы почти закончили. Давайте подведем некоторые итоги.

Обзор главы

Этот обстоятельный разговор был посвящен глубокому изучению конкурентной работы React, затрагивающей множество аспектов, включая Fiber-согласователь, планирование, отсрочку обновлений, полосы рендеринга и новые хуки — `useTransition` и `useDeferredValue`.

Мы начали с обсуждения Fiber-согласователя, ядра механизма конкурентного рендеринга React. Именно он дает возможность фреймворку разбивать работу на более мелкие фрагменты и управлять приоритетом выполнения, что позволяет React быть "прерываемым" и поддерживать конкурентный рендеринг. Это в значительной степени повышает способность React плавно работать со сложными высокопроизводительными приложениями, гарантируя, что взаимодействие с пользователем остается отзывчивым даже во время интенсивных вычислений.

Затем мы перешли к концепции планирования и отсрочки обновлений, которая, по сути, позволяет React устанавливать приоритет одних обновлений состояния над другими. React может откладывать обновления с более низким приоритетом в пользу обновлений с более высоким приоритетом, тем самым обеспечивая бесперебойную работу пользователей даже при большой нагрузке. В качестве примера было приведено приложение для чата, в котором обновления входящих сообщений были запланированы разумно и отображались без блокировки пользовательского интерфейса.

Далее обсуждение перешло к полосам рендеринга, центральной концепции конкурентных особенностей React. Полосы рендеринга — это механизм, который React использует для присвоения приоритета обновлениям и эффективного управления их выполнением. В этом заключен секрет того, как React решает, какие обновления являются срочными и требуют немедленной обработки, а какие можно отложить на более поздний срок. В подробном объяснении говорилось о том, как для эффектив-

ной обработки нескольких приоритетов полосы рендеринга используют растровые маски.

Затем мы углубились в изучение новых хуков, введенных для конкурентных операций в React, а именно `useTransition` и `useDeferredValue`. Эти хуки предназначены для обработки переходов и обеспечения более плавного взаимодействия с пользователем, особенно при выполнении операций, которые занимают значительное количество времени.

Сначала был рассмотрен механизм `useTransition`, который позволяет реагировать на переход между состояниями таким образом, чтобы обеспечить отзывчивый пользовательский интерфейс, даже если для подготовки нового состояния требуется некоторое время. Другими словами, он позволяет отложить обновление до следующего цикла рендеринга, если компонент в данный момент отображается.

Мы также обсудили механизм `useDeferredValue`, который откладывает обновление менее важных частей компонента, тем самым предотвращая неудобства при работе с пользователем. По сути, он позволяет React "удерживать" предыдущее значение немного дольше, если подготовка нового значения занимает слишком много времени.

Наконец, мы углубились в проблемы конкуренции в React, включая разрыв, и изучили, как `useSyncExternalStore` может помочь поддерживать согласованность состояния при нескольких конкурентных отрисовках.

На протяжении всего разговора повторяющейся темой было объяснение того, "что" и "почему" стоит за стратегиями React по управлению сложными, динамичными приложениями с большими объемами вычислений и как разработчики могут использовать эти стратегии для обеспечения плавного и отзывчивого взаимодействия с пользователями.

Проверьте ваши знания

Давайте зададим себе несколько вопросов, чтобы проверить наше понимание концепций, изложенных в этой главе:

1. Что такое Fiber-согласователь React и как он способствует работе со сложными высокопроизводительными приложениями?
2. Объясните концепцию планирования и отсрочки обновлений в React. Как это помогает поддерживать бесперебойную работу пользователей даже при большой нагрузке?
3. Что такое полосы рендеринга в React и как они управляют выполнением обновлений? Можете ли вы описать, как полосы рендеринга используют растровые маски для обработки нескольких приоритетов?
4. Какова цель использования хуков `useTransition` и `useDeferredValue` в React? Опишите ситуацию, в которой каждый из них был бы полезен.
5. Когда может быть неуместно использовать `useDeferredValue`? Каковы некоторые компромиссы, связанные с использованием этого хука?

Что дальше?

Теперь, когда у вас есть глубокое понимание конкурентных особенностей React и его внутренней работы, вы хорошо подготовлены к тому, чтобы в полной мере использовать его потенциал при создании высокопроизводительных приложений. В *главе 8* мы рассмотрим различные популярные фреймворки, созданные поверх React, такие как Next.js и Remix, которые еще больше упрощают процесс разработки, предоставляя лучшие практики, соглашения и дополнительные функции.

Эти фреймворки предназначены для того, чтобы помочь вам с легкостью создавать сложные приложения, решая многие общие проблемы, такие как серверный рендеринг, маршрутизация и фрагментация кода. Используя достоинства этих фреймворков, вы сможете сосредоточиться на расширении возможностей вашего приложения, обеспечивая при этом оптимальную производительность и удобство для пользователей.

Впереди вас ждет подробное знакомство с этими мощными платформами. Вы узнаете, как создавать масштабируемые, производительные и многофункциональные приложения, используя React и его экосистему.